Week 3 · Records and data

Programming for Information Systems

One record per offer, one ID per record

Class warm-up · Research 2 exchange

In groups of three, share the rule you researched.

1. Read your rule aloud, **exactly**. Can the group find a case it does not cover?
2. Share your **boundary** case. Does everyone agree what happens exactly at the limit?
3. Share your **odd** case (missing, negative, unusual). What should the system do?

> 📝 Write down one edge case from a classmate.

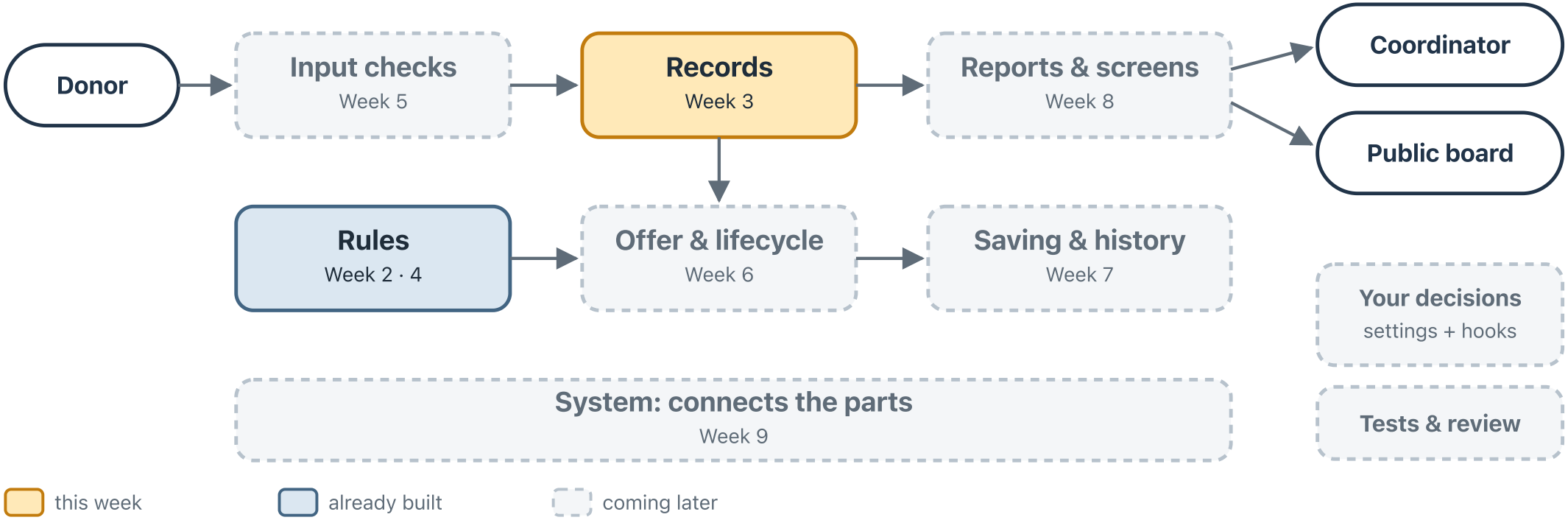
This week

Question	How does a system remember many things and tell them apart?
Python	lists, dictionaries, a list of dictionaries, `for`, the accumulator, finding by ID
Principle 3	One record per real thing: one record, one unique ID, no fact stored twice
You build	`total_portions`, `find_offer` (⭐⭐ `portions_by_category`)

Your system, by Week 10

Your FoodBridge system

Week 3: keep one record per offer and add them up



The idea: records and IDs

Think of a **class list**.

- Each student is **one row**. Every row has the same columns.
- Two students can share a name, but **never** a student ID.
- The office looks you up by **ID**, not by name.

> In our system each offer is one **record** with the same **fields** and one unique **ID** ("F001"). The list of fields is the **data model**.

The data model: which fields belong?

Field	Example	Why we keep it
`id`	`"F001"`	tell offers apart, even with the same food
`donor`, `contact`	`"Sunrise Bakery"`, `"sunrise@example.org"`	thank and call back (contact is private)
`food`, `category`	`"Bread rolls"`, `"bread"`	what it is; category decides if we accept it
`portions`, `hours_left`	`30`, `5`	how much, and how soon
`status`, `partner`	`"approved"`, `"Hope Children's Centre"`	where it is, who receives it

- **Not** in the record: "total portions today". Totals are **calculated** when needed.
- Why an ID, not the food name? Food names repeat and get corrected. IDs never do.

Python foundations

Lists, dictionaries and loops

A list keeps values in order

```
volunteers = ["Dara", "Sophea", "Vanna"]
print(volunteers[0])          # Dara: positions start at 0
print(len(volunteers))        # 3
volunteers.append("Kosal")    # add one at the end
print("Sophea" in volunteers) # True
# print(volunteers[4])        # IndexError
```

- The position number is the **index**. The first index is 0.

A dictionary gives each value a name

```
item = {"name": "Notebook", "price_riel": 4000, "in_stock": 25}
print(item["price_riel"])            # 4000
item["price_riel"] = 3500            # change a value
item["colour"] = "blue"              # add a new key
print(item.get("size"))              # None, no error
print(item.get("size", "one size"))  # a default you choose
```

- Each pair is **key: value**. Look up by key, not by position.
- `item["size"]` would stop with a `KeyError`; `.get` asks politely.

A record is a dictionary; a day is a list

```
offers = [
    {"id": "F001", "food": "Bread rolls", "portions": 30},
    {"id": "F002", "food": "Fried rice boxes", "portions": 18},
]
print(len(offers))           # 2
print(offers[1]["food"])     # Fried rice boxes
```

- `offers[1]["food"]`: first the **position** in the list, then the **key** in the record.
- Almost every information system has this shape.

`for`: the same thing for every item

```
for offer in offers:
    print(offer["id"], offer["food"], offer["portions"], "portions")
```

- The indented lines run **once per item**.
- `offer` points to the current item.
- Habit: a **plural** name for the list, a **singular** name for one item.

The accumulator: start at zero, add each

```
prices_riel = [4000, 1500, 2000]
total_riel = 0                      # 1. start before the loop
for price_riel in prices_riel:
    total_riel += price_riel        # 2. add each item
print(total_riel)                   # 3. use it after the loop
```

- `+=` means "add to what is already there".
- The same pattern **counts**: `count = 0`, then `count += 1`.

🤔 What if `total_riel = 0` were **inside** the loop?

Find a record by its ID

```
def find_item(items, wanted_code):
    for item in items:
        if item["code"] == wanted_code:
            return item          # found: stop and send it back
    return None                  # checked every item: not found
```

- `return` stops the loop at once.
- `return None` lines up with `for`: you can only say "not found" **after** looking at everything.

⚖️ Good or risky?

Version A (from an AI assistant)

```
offers_by_food = {}
offers_by_food["Bread rolls"] = {
    "donor": "Sunrise Bakery",
    "portions": 30}
offers_by_food["Bananas"] = {
    "donor": "Green Market Stall",
    "portions": 40}
stored_total = 70
# later, a second bakery:
offers_by_food["Bread rolls"] = {
    "donor": "Golden Oven Bakery",
    "portions": 12}
stored_total += 12
```

Version B

```
offer_records = []
offer_records.append({"id": "F001",
    "donor": "Sunrise Bakery",
    "portions": 30})
offer_records.append({"id": "F002",
    "donor": "Green Market Stall",
    "portions": 40})
offer_records.append({"id": "F003",
    "donor": "Golden Oven Bakery",
    "portions": 12})
total = 0
for record in offer_records:
    total += record["portions"]
```

Principle 3 · One record per real thing

- **One record** for each real offer, with **one unique ID**.
- Choose keys that never repeat: an ID, not a food name.
- **Do not store the same fact twice**: calculate totals from the records.

> When you review AI code, ask: *what happens when two things have the same name?*

You build: totals and search

```
def total_portions(records):
    # start at 0, add each record's "portions", return the total
    return None

def find_offer(records, offer_id):
    # return the matching record; after the loop, return None
    return "?"

bridge.check("total_portions", total_portions)  # 100, and [] → 0
bridge.check("find_offer", find_offer)       # F002 found, F999 → None
```

Plug in: compare with the real system

```
my_system = bridge.System(MY_SETTINGS)
my_system.run_day(show=False)
system_records = my_system.records()

print(total_portions(system_records))              # yours
print(my_system.numbers()["portions_offered"])    # the system's
print(find_offer(system_records, "F002"))
```

- Two independent ways, one answer: trust both a little more.

🧭 Your decision · which categories?

```
MY_SETTINGS["CATEGORIES"] = ["bread", "rice meals", "fruit",
                             "vegetables", "snacks", "soup"]
```

	Default categories	+ "soup"
accepted at the door	4 offers	5 offers
portions offered	100	125
refused at the door	5	4

- The noodle soup (25 portions, 3 hours left) goes to the **top** of the list.
- Write categories in **lowercase**: `"soup"`, not `"Soup"`.

This week's tasks

- ⭐ **Core:** P1–P4, three problems in Version A, `total_portions` and `find_offer` with all ✅, matching the system, decision log for `CATEGORIES`.
- ⭐⭐ **Stretch:** S1 count by category; `portions_by_category` with ✅.
- No new research this week. Research 2 deadline as announced in class.

Next week · Functions you can trust · Midterm

Your functions work on the sample. Will they work on **every** input? You will write **tests** that prove it. Session 3 is the individual **midterm**.